PS 1

AUTHOR
Ben Schiffman

PUBLISHED
October 5, 2001

1. **PS1:** Due Sat Oct 5 at 5:00PM Central. Worth 50 points. Initiate your [repo]

We use (`*`) to indicate a problem that we think might be time consuming.

Steps to submit (5 points on PS1 and 10 points on PS2)

1. "This submission is my work alone and complies with the 30538 integrity policy." Add your initials to indicate your agreement: **__**
2. "I have uploaded the names of anyone I worked with on the problem set [here]" **__** (1 point)
3. Late coins used this pset: **__** Late coins left after submission: **__**
4. Knit your `ps1.qmd` to make `ps1.pdf`.
   - The PDF should not be more than 25 pages. Use `head()` and re-size figures when appropriate.
5. Push `ps1.qmd` and `ps1.pdf` to your github repo. It is fine to use Github Desktop.
6. Submit `ps1.pdf` via Gradescope (4 points)
7. Tag your submission in Gradescope

Read in one percent sample (15 Points)

1.

```
import pandas as pd
import os
import time


pset_dpath = r"/Users/benschiffman/Desktop/Python 2/ppha30538_fall2024/problem_sets/ps1/d

oneperc_path = r"parking_tickets_one_percent.csv"

start_time = time.time()
df = pd.read_csv(os.path.join(pset_dpath, oneperc_path))
end_time = time.time()
total_time = end_time - start_time

print("it takes", total_time, "seconds to read this file")
print(len(df))
assert len(df) == 287458, "Well, try again"
```

```
it takes 0.5829911231994629 seconds to read this file
287458

/var/folders/yb/wxm346513_x57339c0hyc5280000gn/T/ipykernel_29020/3155636270.py:11:
DtypeWarning: Columns (7) have mixed types. Specify dtype option on import or set
```

```
low_memory=False.
  df = pd.read_csv(os.path.join(pset_dpath, oneperc_path))
```

it takes 0.581108808517456 seconds to read this file. 1.

```
size = os.path.getsize(os.path.join(pset_dpath, oneperc_path))
print(size/1000000)
```

```
83.655348
```

The data dataset is 83.655348 MB 1.

```
df.head()
df.tail()

top_data = df[:500]

top_data["issue_date"] = pd.to_datetime(top_data["issue_date"])

#this function compares dates in a column to see if they are in chronological order by in
def is_sorted(data):
  sorted = False
  for date in data:
    baseline = data[0]
    if date >= baseline:
      baseline = date
    else:
      return print("The column is not sorted")
  return print("The column is sorted")

is_sorted(top_data["issue_date"])
```

```
The column is sorted

/var/folders/yb/wxm346513_x57339c0hyc5280000gn/T/ipykernel_29020/2682147371.py:6:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  top_data["issue_date"] = pd.to_datetime(top_data["issue_date"])
```

It seems like the data is sorted by issue date and time. And the column is sorted.

Cleaning the data and benchmarking (15 Points)

1.

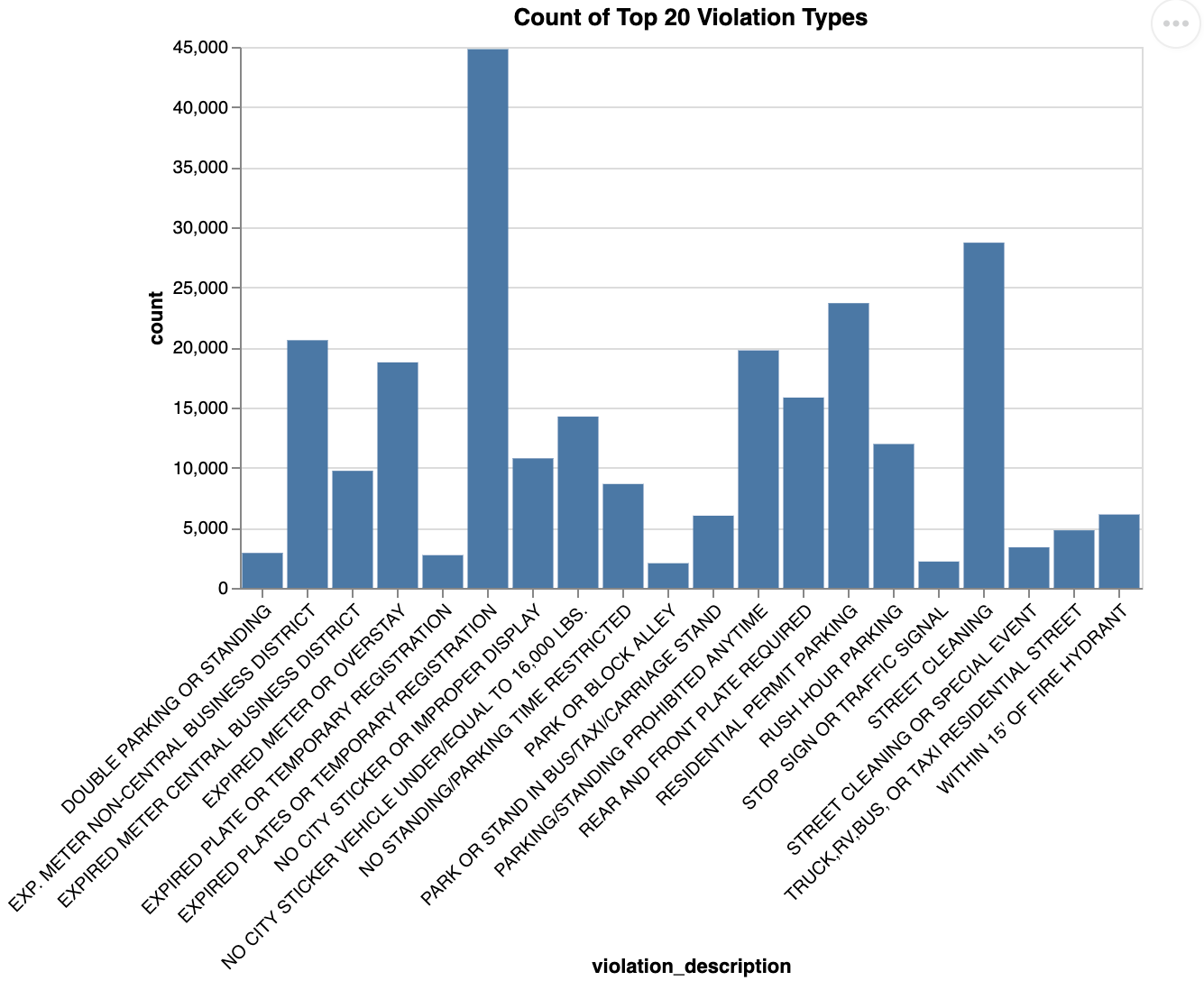
```
df["issue_date"] = pd.to_datetime(df["issue_date"])
df_2017 = df[df["issue_date"].dt.year == 2017]
ct = len(df_2017)

year_ct = ct * 100
year_ct
```

```
2236400
```

There are 22364 tickets in the 1 percent dataset, which implies there were ~2,236,400 tickets issued in 2017. The article states that illinois issues more than 3 million tickets a year. This indicates that 2017 is a lower ticket year, but we need a lot more information to make much more of a statement than that.

1.

```
import altair as alt

top_violations = df.groupby("violation_description").size()
top_violations = top_violations.sort_values(ascending=False).head(20)
top_violations = top_violations.reset_index()
top_violations.columns = ["violation_description", "count"]

chart = alt.Chart(top_violations).mark_bar().encode(
    alt.X("violation_description", axis=alt.Axis(labelAngle=-45, labelLimit = 400)),
    alt.Y("count")
).properties(
    width=500,
    title = "Count of Top 20 Violation Types"
)
chart

#ChatGPT was used in this cell for ideation on how to sort data into top 20 violation typ
```



Visual Encoding (15 Points)

1.

```
pd.set_option('display.max_columns', None)
df.head()
```

	Unnamed: 0	ticket_number	issue_date	violation_location	license_plate_number
0	1	51482901.0	2007-01-01 01:25:00	5762 N AVONDALE	d41ee9a4cb0676e641399ad14aaa20d06f2c6896d
1	2	50681501.0	2007-01-01 01:51:00	2724 W FARRAGUT	3395fd3f71f18f9ea4f0a8e1f13bf0aa15052fc8e560

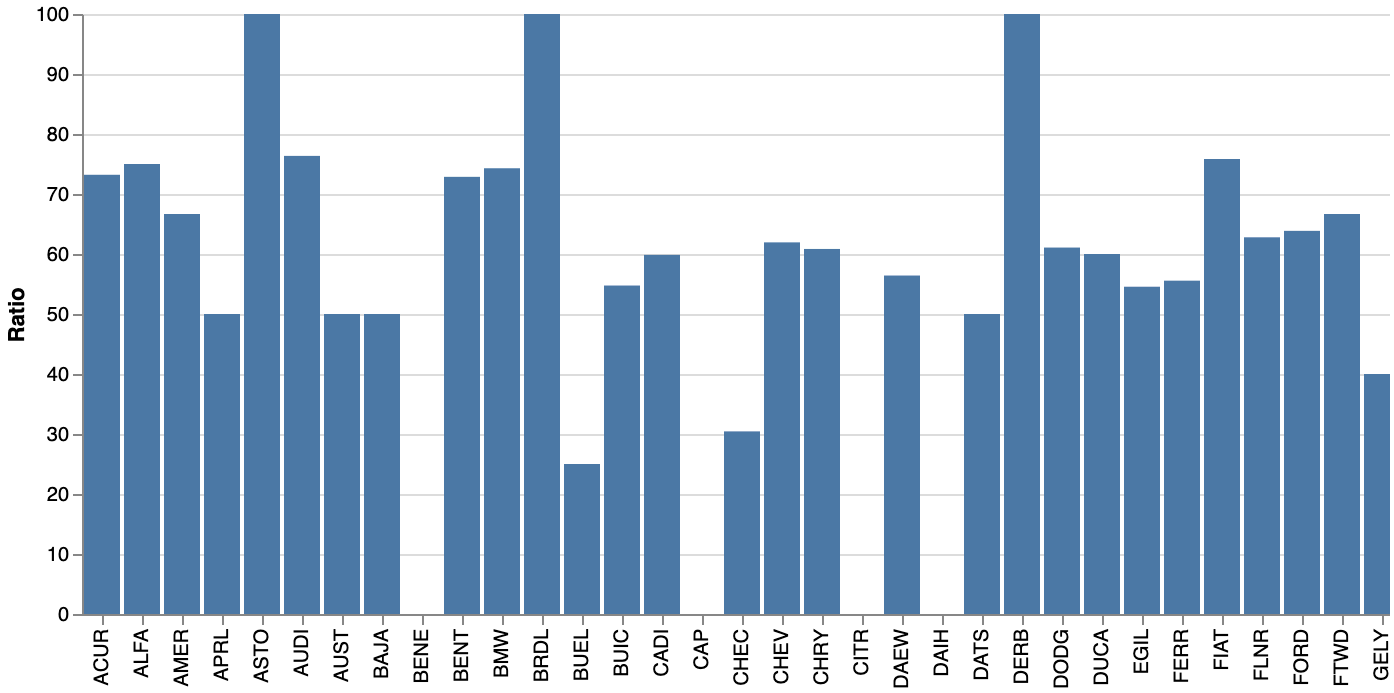
	Unnamed: 0	ticket_number	issue_date	violation_location	license_plate_number
2	3	51579701.0	2007-01-01 02:22:00	1748 W ESTES	302cb9c55f63ff828d7315c5589d97f1f8144904d6
3	4	51262201.0	2007-01-01 02:35:00	4756 N SHERIDAN	94d018f52c7990cea326d1810a3278e2c6b1e8b44
4	5	51898001.0	2007-01-01 03:50:00	7134 S CAMPBELL	876dd3a95179f4f1d720613f6e32a5a7b86b0e6f98

Coloumn	Data Type
Ticket Number	Ordinal
Issue Date	Temporal
Violation Location	Nominal
LP Number	Nominal
LP State	Nominal
LP Type	Nominal
ZIP Code	Nominal
Vio Code	Nominal
Vio Description	Nominal
Unit	Nominal
Unit Description	Nominal
Vehicle Make	Nominal
Fine Amount 1	Quantitative
Fine Amount 2	Quantitative
Amount Due	Quantitative
Total Payment	Quantitative

Coloumn	Data Type
Ticket Queue	Ordinal
TQ Date	Temporal
Notice Level	Ordinal
Hearing Disp	Nominal
Notice Num	Nominal
Officer	Nominal
Address	Nominal

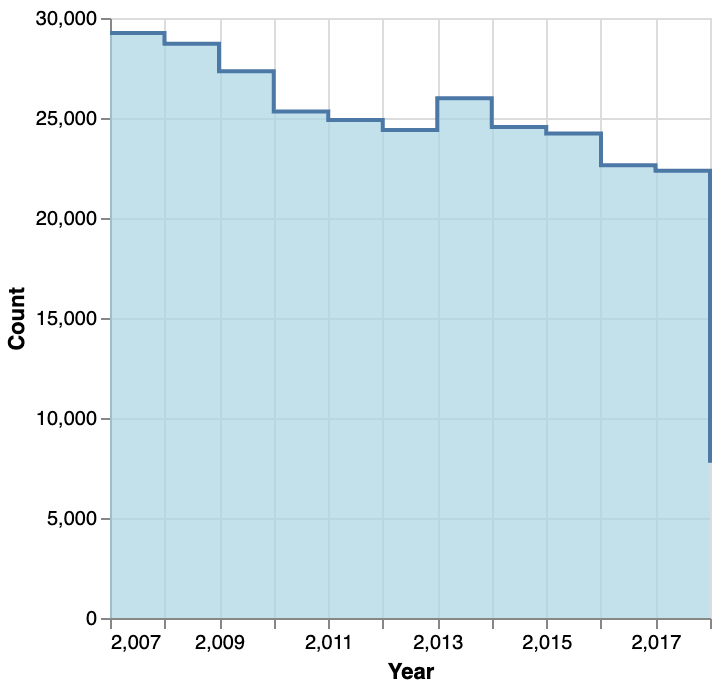
1.

```
df["Paid"] = df["ticket_queue"].apply(lambda x: 1 if x == "Paid" else 0)
make_paid = df.groupby(["vehicle_make", "Paid"]).size()
make_paid = make_paid.reset_index(name = "count")
pivot = make_paid.pivot(index = "vehicle_make", columns = "Paid", values = "count").filln
pivot.columns = ["Unpaid", "Paid"]
pivot.reset_index(inplace=True)
pivot

pivot["Ratio"] = pivot["Paid"]/(pivot["Unpaid"] + pivot["Paid"]) * 100
pivot

paid_chart = iol_chart = alt.Chart(pivot).mark_bar().encode(
    alt.X("vehicle_make"),
    alt.Y("Ratio")
).properties(
    title = "Percent of Tickets Paid by Make"
)
paid_chart

#chatGPT utilized for pivot functionality on this problem. I used a number of queries to
```



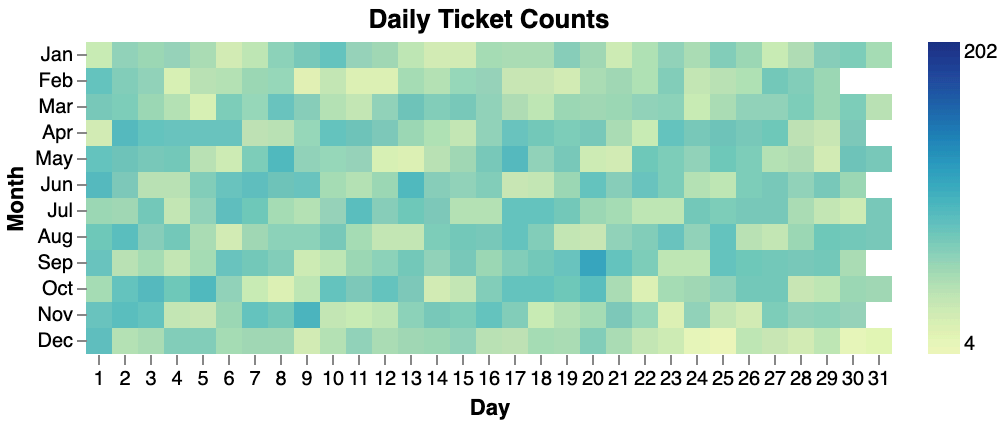
1.

```
import altair as alt
import datetime

df
df["issue_date"] = pd.to_datetime(df["issue_date"])
df["Year"] = df["issue_date"].dt.year
year_tix = df.groupby("Year").size().reset_index(name = "Count")
year_tix.head()

alt.Chart(year_tix).mark_area(
    color="lightblue",
    interpolate='step-after',
    line=True
).encode(
    x='Year',
    y='Count'
)
```



1.

```
import altair as alt

ticket_counts = df.groupby(df["issue_date"].dt.date).size().reset_index(name="count")
ticket_counts["issue_date"] = pd.to_datetime(ticket_counts["issue_date"])

chart2 = alt.Chart(ticket_counts, title = "Daily Ticket Counts").mark_rect().encode(
    alt.X("date(issue_date):O").title("Day").axis(format="%e", labelAngle=0),
    alt.Y("month(issue_date):O").title("Month"),
    alt.Color("count").title(None),
    tooltip=[
        alt.Tooltip("monthdate(issue_date)", title="Date"),
        alt.Tooltip("count", title="Tickets Count"),
    ],
).configure_view(
    step=13,
    strokeWidth=0
).configure_axis(
    domain=False
)
chart2

#I used chatGPT for error handling, specifically for the error "TypeError: Object of type
```

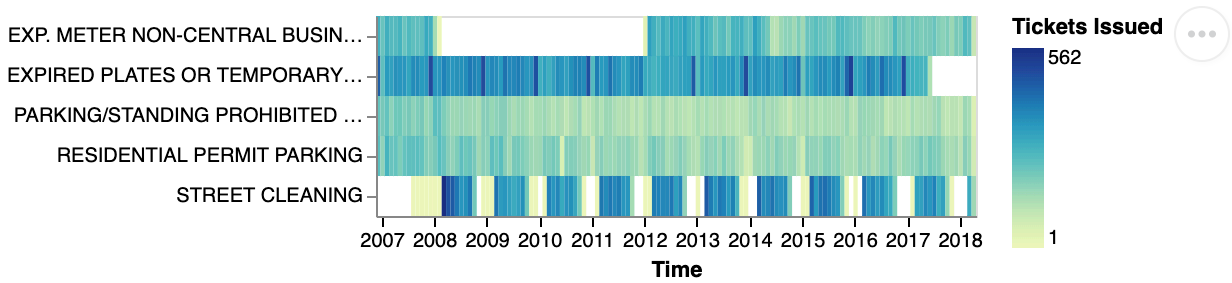


1.

```
import altair as alt
import datetime as dt

#first identify top 5 violation types
top5_violations = df.copy()
top5_violations = top5_violations.groupbytop5_violations = top5_violations.groupby("viola
top5_violations = top5_violations.sort_values(by = "count", ascending=False).head(5)
top5_violations

#create a new df with just top 5
lasagna_df = pd.merge(df, top5_violations, on = "violation_description", how = "right")
lasagna_df["issue_date"] = pd.to_datetime(lasagna_df["issue_date"])
lasagna_df['year_month'] = lasagna_df['issue_date'].dt.to_period('M')
lasagna_df

#summarize data becuase of too many rows
#ChatGPT aided with this step. I used the prompt "How to deal with max rows error altair"

monthly_counts = lasagna_df.groupby(["violation_description",'year_month']).size().reset_
monthly_counts["year_month"] = monthly_counts["year_month"].dt.to_timestamp()
monthly_counts

#create graph
color_condition = alt.condition(
    "month(datum.value) == 1 && date(datum.value) == 1",
    alt.value("black"),
    alt.value(None),
)
alt.Chart(monthly_counts, width=300, height=100).transform_filter(
    alt.datum.symbol != "GOOG"
).mark_rect().encode(
    alt.X("yearmonthdate(year_month):O")
        .title("Time")
        .axis(
            format="%Y",
            labelAngle=0,
            labelOverlap=False,
```

```
            labelColor=color_condition,
            tickColor=color_condition,
        ),
    alt.Y("violation_description:N").title(None),
    alt.Color("sum(count)").title("Tickets Issued")
)
```

```
/var/folders/yb/wxm346513_x57339c0hyc5280000gn/T/ipykernel_29020/383925509.py:6:
UserWarning: Pandas doesn't allow columns to be created via a new attribute name - see
https://pandas.pydata.org/pandas-docs/stable/indexing.html#attribute-access
  top5_violations = top5_violations.groupbytop5_violations =
top5_violations.groupby("violation_description").size().reset_index(name = "count")
```



1. The prior three questions created the filled step, heat map, and lasagna charts. The filled step shows broader timeseries data, which is best for showing how one data point changes over time. The heat map chart is going to be best for comparing patterns over the course of weeks and days in a month as well as over the course of a year, aggreggated from many years of data. This is best for seeing a pattern within a calandar year for one data point, perhaps to establish a baseline pattern to compare an individual year to. Finally the lasagna chart compares patterns over time between multiple categories. This is best for comparing categories with eachother within the same time frame, and can show some basic patterns within each category.
2. To show a reader that distributions are not evenly distributed over time, I would show the heat map. This shows the patterns in distribution by day and month. That said, both other graphs show different time periods in different detail, so they have their own benefits. The heat map simply shows differences in distribution of all tickets over at the course of the year. The filled step graph shows a broad pattern, but may be due to external factors like policy changes, and the lasagna map may on the other hand show too small a sample of types of tickets to assume a wider pattern.